100 Most Asked Python Interview QnA

Made By:

@python_world_in
+91 9771025313 Whatsapp

1. **What is Python?**
   Python is a high-level, interpreted programming language known for its simplicity and readability. It emphasizes code readability and encourages a clean and concise coding style.
2. **What are the key features of Python?**
   Key features of Python include its easy-to-read syntax, dynamic typing, automatic memory management, extensive standard library, and support for multiple programming paradigms.
3. **How is Python different from other programming languages?**
   Python stands out with its simplicity, readability, and easy-to-understand syntax. It has a large and active community, extensive libraries, and is widely used in various domains such as web development, data analysis, and scientific computing.
4. **What is PEP 8?**
   PEP 8 is the official style guide for Python code. It provides guidelines on how to format Python code to enhance readability and maintain consistency across projects.
5. **What are Python modules?**
   Python modules are files containing Python code that define functions, classes, and variables. They allow code reuse and organization, making it easier to manage and maintain larger projects.
6. **What is a Python package?**
   A Python package is a way to organize related modules into a directory hierarchy. It allows for a logical grouping of modules, making it easier to manage and distribute code.
7. **How do you comment in Python?**
   Comments in Python are denoted by the # character. Anything after the # is considered a comment and is ignored by the Python interpreter.
8. **What are Python data types?**
   Python supports various data types, including integers, floating-point numbers, strings, lists, tuples, dictionaries, and booleans. Each data type has its own characteristics and uses.
9. **What is type conversion in Python?**
   Type conversion, also known as type casting, is the process of converting one data type into

another. Python provides built-in functions like int(), float(), str(), etc., to perform type conversion.

10. **What is string interpolation in Python?**
String interpolation in Python allows you to embed expressions or variables within a string, making it easier to construct dynamic strings. It can be done using f-strings or the format() method.
11. **What are Python conditional statements?**
Python conditional statements, such as if, elif, and else, allow you to perform different actions based on certain conditions. They control the flow of the program based on the truthfulness of the conditions.
12. **What are Python loops?**
Python loops, like for and while, enable you to execute a block of code repeatedly. They iterate over a sequence or execute until a specific condition is met.
13. **What is the difference between range() and xrange() in Python 2?**
In Python 2, range() generates a list of numbers, while xrange() returns an iterator. xrange() is more memory-efficient for large ranges because it generates values on the fly.
14. **What are Python functions?**
Python functions are reusable blocks of code that perform a specific task. They help in code organization, reusability, and modularity. Functions can accept arguments and return values.
15. **What is the difference between a function and a method in Python?**
In Python, a function is a standalone block of code that can be called independently. A method, on the other hand, is a function that is associated with an object or a class and can access the object's data.
16. **How do you define a function in Python?**
A function in Python is defined using the def keyword, followed by the function name, parentheses for parameters (if any), and a colon. The function body is indented below.
17. **What is the __init__ method used for?**
The __init__method is a special method in Python classes that is automatically called when an object is created from the class. It is used to initialize the object's attributes and perform setup tasks.
18. **What is object-oriented programming (OOP)?**
Object-oriented programming (OOP) is a programming paradigm that organizes code into objects, which are instances of classes. It emphasizes encapsulation, inheritance, and polymorphism.
19. **What are Python classes and objects?**
In Python, a class is a blueprint that defines the properties and behaviors of objects. An object is an instance of a class. It represents a specific entity and can interact with other objects.
20. **How do you create an object in Python?**
An object is created by calling the class as if it were a function. The class acts as a constructor, initializing the object and returning it.

21. **What is inheritance in Python?**
Inheritance is a mechanism in Python that allows a class to inherit properties and methods from another class. It enables code reuse and supports the creation of hierarchical class structures.
22. **What is method overriding?**
Method overriding is the process of defining a method in a subclass that has the same name as a method in its superclass. The subclass method overrides the implementation of the superclass method.
23. **What is method overloading?**
Method overloading is not directly supported in Python. However, you can achieve similar functionality by defining a single method with default argument values or using variable-length arguments.
24. **What is encapsulation in Python?**
Encapsulation is the process of bundling data and methods together within a class. It allows for data hiding and controlling access to the object's attributes using getter and setter methods.
25. **What is polymorphism in Python?**
Polymorphism is the ability of an object to take on multiple forms or have multiple behaviors. In Python, polymorphism is achieved through method overriding and method overloading (using default argument values or variable-length arguments).
26. **What is a generator in Python?**
A generator in Python is a function that returns an iterator. It allows you to generate a sequence of values on-the-fly, conserving memory and improving performance.
27. **What are decorators in Python?**
Decorators are a way to modify the behavior of a function or class without directly changing its source code. They are defined using the @decorator_name syntax and can be used for tasks like logging, timing, or modifying function arguments.
28. **What is a lambda function in Python?**
A lambda function is an anonymous function in Python that is defined using the lambda keyword. It is a shorthand way to create small, one-line functions without explicitly defining a function using def.
29. **What is a module in Python?**
A module in Python is a file containing Python definitions and statements. It can be imported and used in other Python programs to access its functions, classes, and variables.
30. **How do you import modules in Python?**
Modules can be imported in Python using the import keyword followed by the module name. You can also import specific objects from a module using the from module_name import object_name syntax.
31. **What is a virtual environment in Python?**
A virtual environment in Python is a self-contained directory that contains a specific version

of Python interpreter and installed packages. It allows you to isolate Python environments for different projects and manage their dependencies.

32. **What are exceptions in Python?**

Exceptions in Python are events that occur during the execution of a program that disrupt the normal flow of the code. They can be handled using try-except blocks to gracefully handle errors and exceptions.

33. **What is error handling in Python?**

Error handling in Python involves using try-except blocks to catch and handle exceptions that may occur during the execution of the code. It allows for graceful recovery from errors and prevents the program from crashing.

34. **What is the purpose of the try-except-else-finally block in Python?**

The try-except-else-finally block in Python is used for exception handling. The try block contains the code that may raise an exception. The except block is used to handle specific exceptions. The else block is executed if no exceptions occur. The finally block is always executed, regardless of whether an exception occurred or not.

35. **What are the built-in data structures in Python?**

Python provides several built-in data structures, including lists, tuples, dictionaries, sets, and strings. These data structures offer different ways to store, manipulate, and retrieve data.

36. **What is a list in Python?**

A list in Python is an ordered collection of items that can be of different data types. It is mutable, meaning its elements can be modified. Lists are denoted by square brackets [ ] and can contain elements separated by commas.

37. **What is a tuple in Python?**

A tuple in Python is an ordered collection of items similar to a list. However, tuples are immutable, meaning their elements cannot be changed once assigned. Tuples are denoted by parentheses ( ) and can contain elements separated by commas.

38. **What is a dictionary in Python?**

A dictionary in Python is an unordered collection of key-value pairs. It is mutable and allows fast access to values based on their associated keys. Dictionaries are denoted by curly braces { } and use colons : to separate keys and values.

39. **What is a set in Python?**

A set in Python is an unordered collection of unique elements. It is mutable and provides mathematical set operations like union, intersection, and difference. Sets are denoted by curly braces { } or the set() function.

40. **What is a string in Python?**

A string in Python is a sequence of characters enclosed in single quotes, double quotes, or triple quotes. It is immutable, meaning its individual characters cannot be changed. Strings can be manipulated and operated upon in various ways.

41. **How do you concatenate strings in Python?**

Strings can be concatenated in Python using the + operator or by using the .join() method.

The + operator concatenates two strings, while the .join() method concatenates multiple strings using a specified delimiter.

42. **How do you format strings in Python?**

Strings can be formatted in Python using the % operator, the str.format() method, or f-strings (formatted string literals). These methods allow you to insert values into placeholders within a string.

43. **What are file handling operations in Python?**

File handling operations in Python involve reading from and writing to files. Python provides built-in functions and methods to open, read, write, and close files.

44. **How do you open and close a file in Python?**

Files can be opened in Python using the open() function, which takes the file name and the mode of operation as arguments. The close() method is used to close an opened file and free up system resources.

45. **What are the different file modes in Python?**

The different file modes in Python include "r" for reading, "w" for writing (overwriting existing content), "a" for appending, "x" for exclusive creation (fails if the file already exists), and "b" for binary mode.

46. **What is exception handling in file operations?**

Exception handling in file operations involves handling potential errors that may occur while performing file-related operations. This ensures that the program handles file-related exceptions gracefully and avoids crashes or data loss.

47. **What is a context manager in Python?**

A context manager in Python is an object that defines the methods __enter__() and __exit__() to enable the with statement. It allows for resource allocation and deallocation, such as automatically closing a file after use.

48. **What is a generator function in Python?**

A generator function in Python is a special type of function that uses the yield keyword instead of return. It allows you to generate a sequence of values on-the-fly without storing them all in memory at once.

49. **What is a list comprehension in Python?**

A list comprehension in Python is a concise way to create lists based on existing lists or other iterable objects. It allows you to combine looping and conditional logic in a single line of code.

50. **What is the pass statement in Python?**

The pass statement in Python is a placeholder statement that does nothing. It is used as a syntactic placeholder when a statement is required by the Python syntax, but no action is needed.

51. **What is the purpose of the self parameter in Python?**

The self parameter is used as a reference to the current instance of a class in Python. It allows accessing the attributes and methods of that instance within the class definition.

52. **What is the difference between a shallow copy and a deep copy in Python?**
In Python, a shallow copy creates a new object that references the original data, while a deep copy creates a new object with completely independent copies of the original data. Modifying the original data does not affect the deep copy, but it can affect the shallow copy.
53. **What are the advantages of using Python for web development?**
Python offers several advantages for web development, including a wide range of frameworks (such as Django and Flask), a large community, extensive libraries, and easy integration with other technologies.
54. **What is the Global Interpreter Lock (GIL) in Python?**
The Global Interpreter Lock (GIL) is a mechanism in the CPython interpreter (the reference implementation of Python) that allows only one thread to execute Python bytecode at a time. This restricts the parallel execution of Python threads and can impact performance in certain scenarios.
55. **What is a metaclass in Python?**
A metaclass in Python is a class that defines the behavior and structure of other classes. It allows you to customize class creation, modify attributes, and add additional functionality to classes.
56. **How do you handle file I/O errors in Python?**
File I/O errors in Python can be handled using exception handling. By using try-except blocks around file-related operations, you can catch specific exceptions like FileNotFoundError or PermissionError and handle them gracefully.
57. **What is the purpose of the __name__ variable in Python?**
The __name__variable in Python is a built-in variable that represents the current module's name. It can be used to determine whether a module is being run as the main script or imported as a module.
58. **What is the difference between a shallow comparison and a deep comparison in Python?**
In Python, a shallow comparison checks if two objects have the same memory address, while a deep comparison checks if the objects have the same values. Shallow comparisons can be done using the is operator, while deep comparisons are typically done using the == operator.
59. **What are the advantages of using virtual environments in Python?**
Virtual environments in Python provide a dedicated environment for each project, allowing you to isolate project dependencies, avoid conflicts between packages, and maintain project-specific versions of Python and packages.
60. **What is the purpose of the __main__ block in Python?**
The __main__block in Python is used to define the entry point of a Python program. The code inside the if __name__== "__main__": block will only execute if the script is run directly, not when it is imported as a module.
61. **What is the purpose of the __str__ method in Python?**
The __str__method in Python is a special method that returns a string representation of an

object. It is used to provide a human-readable representation of the object when the str() function is called or when the object is printed.

62. **What is the purpose of the __repr__ method in Python?**

The __repr__method in Python is a special method that returns a string representation of an object that can be used to recreate the object. It is used to provide a detailed and unambiguous representation of the object.

63. **What is the difference between the __str__ and __repr__ methods in Python?**

The __str__method is intended to provide a human-readable string representation of an object, while the __repr_method is intended to provide a detailed and unambiguous string representation that can be used to recreate the object.

64. **What is the purpose of the super() function in Python?**

The super() function in Python is used to call a method in a superclass or parent class. It is often used in method overriding to invoke the superclass's implementation of the method before adding additional functionality in the subclass.

65. **What is the purpose of the __getitem__ method in Python?**

The __getitem___method in Python is a special method that allows objects to define behavior for indexing and slicing operations. It is called when an item is accessed using square brackets ([]) and supports accessing items by index or slicing.

66. **What is the purpose of the __setitem__ method in Python?**

The __setitem___method in Python is a special method that allows objects to define behavior for assigning values to items using square brackets ([]). It is called when an item is assigned a value using indexing.

67. **What is the purpose of the __len__ method in Python?**

The __len__method in Python is a special method that returns the length of an object. It is called when the len() function is used on an object.

68. **What is the purpose of the __iter__ method in Python?**

The __iter__method in Python is a special method that returns an iterator object. It is used to make an object iterable, meaning it can be looped over using a for loop or used with other iterator-related functions and constructs.

69. **What is the purpose of the __next__ method in Python?**

The __next___method in Python is a special method that returns the next item in an iterator. It is called by the next() function and is used in conjunction with the __iter_method to create custom iterators.

70. **What is the purpose of the @property decorator in Python?**

The @property decorator in Python is used to define a method as a getter for a class attribute. It allows accessing the attribute as if it were a normal attribute, while internally calling the getter method.

71. **What is the purpose of the @staticmethod decorator in Python?**

The @staticmethod decorator in Python is used to define a static method in a class. Static methods do not require an instance of the class to be called and can be accessed directly from the class itself.

72. **What is the purpose of the @classmethod decorator in Python?**
The @classmethod decorator in Python is used to define a class method. Class methods receive the class itself as the first parameter, allowing them to access and modify class-level attributes and perform operations specific to the class.
73. **What is the purpose of the __call__ method in Python?**
The _call_method in Python is a special method that allows an object to be called as if it were a function. It is called when parentheses are used to invoke the object.
74. **What is the purpose of the *args and **kwargs parameters in Python?**
The *args parameter in Python allows a function to accept a variable number of positional arguments as a tuple, while the **kwargs parameter allows a function to accept a variable number of keyword arguments as a dictionary. This flexibility allows functions to handle different numbers and types of arguments.
75. **What are decorators in Python?**
Decorators in Python are a way to modify or enhance the behavior of functions or classes without directly modifying their source code. Decorators are implemented as functions that wrap around the target function or class and add additional functionality.
76. **What is the purpose of the @classmethod decorator in Python?**
The @classmethod decorator in Python is used to define a class method. Class methods receive the class itself as the first parameter, allowing them to access and modify class-level attributes and perform operations specific to the class.
77. **What is a lambda function in Python?**
A lambda function in Python is an anonymous function that can be defined in a single line. It is often used for simple, one-time operations and does not require a formal def statement.
78. **What are modules in Python?**
Modules in Python are files that contain Python code and definitions. They can be imported and used in other Python programs to provide reusable functionality.
79. **What are packages in Python?**
Packages in Python are a way to organize related modules into a directory hierarchy. They allow for better organization and modularization of code, making it easier to manage large projects.
80. **What is the purpose of the __init__.py file in a package?**
The __init__.py file in a package serves as an indicator that the directory is a Python package. It can be empty or contain initialization code that is executed when the package is imported.
81. **What is the purpose of the sys module in Python?**
The sys module in Python provides access to system-specific parameters and functions. It allows interaction with the Python interpreter and provides information about the runtime environment.
82. **What is the purpose of the os module in Python?**
The os module in Python provides a way to interact with the operating system. It allows

performing various operations related to file and directory manipulation, process management, and environment variables.

83. **What is the purpose of the datetime module in Python?**
The datetime module in Python provides classes for manipulating dates and times. It allows creating, formatting, and performing operations on dates and times.

84. **What are decorators in Python?**
Decorators in Python are a way to modify or enhance the behavior of functions or classes without directly modifying their source code. Decorators are implemented as functions that wrap around the target function or class and add additional functionality.

85. **What is the purpose of the @property decorator in Python?**
The @property decorator in Python is used to define a method as a getter for a class attribute. It allows accessing the attribute as if it were a normal attribute, while internally calling the getter method.

86. **What is the purpose of the @staticmethod decorator in Python?**
The @staticmethod decorator in Python is used to define a static method in a class. Static methods do not require an instance of the class to be called and can be accessed directly from the class itself.

87. **What is the purpose of the @classmethod decorator in Python?**
The @classmethod decorator in Python is used to define a class method. Class methods receive the class itself as the first parameter, allowing them to access and modify class-level attributes and perform operations specific to the class.

88. **What is a lambda function in Python?**
A lambda function in Python is an anonymous function that can be defined in a single line. It is often used for simple, one-time operations and does not require a formal def statement.

89. **What are modules in Python?**
Modules in Python are files that contain Python code and definitions. They can be imported and used in other Python programs to provide reusable functionality.

90. **What are packages in Python?**
Packages in Python are a way to organize related modules into a directory hierarchy. They allow for better organization and modularization of code, making it easier to manage large projects.

91. **What is the purpose of the __init__.py file in a package?**
The __init__.py file in a package serves as an indicator that the directory is a Python package. It can be empty or contain initialization code that is executed when the package is imported.

92. **What is the purpose of the sys module in Python?**
The sys module in Python provides access to system-specific parameters and functions. It allows interaction with the Python interpreter and provides information about the runtime environment.

93. **What is the purpose of the os module in Python?**
The os module in Python provides a way to interact with the operating system. It allows

performing various operations related to file and directory manipulation, process management, and environment variables.

94. **What is the purpose of the datetime module in Python?**
The datetime module in Python provides classes for manipulating dates and times. It allows creating, formatting, and performing operations on dates and times.

95. **What is the purpose of the random module in Python?**
The random module in Python provides functions for generating random numbers. It allows you to generate random integers, floating-point numbers, and make random selections from lists.

96. **What is the purpose of the json module in Python?**
The json module in Python provides functions for working with JSON (JavaScript Object Notation) data. It allows encoding Python objects into JSON strings and decoding JSON strings into Python objects.

97. **What is the purpose of the pickle module in Python?**
The pickle module in Python provides functions for serializing and deserializing Python objects. It allows you to convert Python objects into a binary format that can be stored or transmitted, and then restore them back into objects.

98. **What are generators in Python?**
Generators in Python are functions that can be paused and resumed, allowing them to produce a sequence of values over time. They are memory-efficient and provide a convenient way to iterate over large or infinite sequences.

99. **What is the purpose of the yield keyword in Python?**
The yield keyword in Python is used in the context of generators. It allows a generator function to temporarily pause and yield a value to the caller, without losing its internal state. The generator can then be resumed to continue execution from where it left off.

100. **What is the purpose of the zip() function in Python?**
The zip() function in Python is used to combine multiple iterables (such as lists or tuples) into a single iterable of tuples. It pairs up corresponding elements from each iterable, stopping when the shortest iterable is exhausted.